Chapter 2: Background & Literature review

Optimizing playlists flow and sequencing: A model to find the best path across a set of songs while maintaining a predefined “energetic arc” and ensuring high quality local transitions

Yuval Antman, Ohad Motola, Shelly Kritsberg, Tomer Filo

Gap Analysis & Proposed Solution: What We Do Differently

Most traditional music recommendation systems (like Collaborative Filtering or Content-Based Filtering) focus on recommending individual tracks based on semantic similarity or past user interactions, often ignoring the temporal context and the holistic listening experience. While automated DJ systems and playlist generators have been explored in the past, they typically suffer from fundamental limitations: they either focus strictly on local transition mechanics without a long-term structural goal, or they rely on generic Reinforcement Learning (RL) aiming to minimize track skips. As Spotify Researchers (Tomasi et al. 2023 Abstract section) note, conventional techniques frequently suffer from a **"misalignment between offline model objectives and online user satisfaction metrics"** [1], failing to guarantee a cohesive musical narrative**.**

Our project approaches playlist generation as a **sequential optimization problem** rather than as a one-time static ordering task or a recommendation problem over a large music catalog. Instead of computing a full graph over the entire track pool and then solving for one fixed path, given a fixed pool of songs, a start track, an end track, and a predefined Energy Arc, the goal is to **reorder the songs into a coherent musical sequence**. The sequence should follow the **desired global energy structure**, such as a gradual build-up or a double-peak shape, **while also keeping adjacent songs musically compatible** in terms of tempo, harmonic relation, energy similarity, and acoustic features.
To avoid the limitations of purely greedy sequencing, the proposed method uses **recursive bidirectional Beam Search strategy.** The algorithm is inspired by bidirectional heuristic search, especially the **MM algorithm** proposed by Holte et al. (2016) [2], which runs searches from both the start and the goal and is designed to meet near the middle of the solution path. In our case, this idea is adapted to playlist sequencing: The system grows candidate partial sequences from the opening track and from the closing track until reaching the positions **around the middle of the playlist**. It then selects a **middle anchor** song according to a cost function that combines **transition quality, energy similarity to the neighboring candidates**, **and distance from the target Energy Arc at the middle position**, with stronger emphasis on the Energy Arc at this stage.
After the middle song is selected, it is **fixed in place** and the **problem is split into two smaller subproblems**: sequencing the left part from the start track to the middle track, and sequencing the right part from the middle track to the end track. The same bidirectional process is then applied recursively to each subsegment. A

global memory of fixed and already-used songs is maintained throughout the process, so that no song can be selected more than once. **When a subsegment becomes small, the algorithm switches from Beam Search to direct local optimization**, giving stronger weight to transition smoothness between adjacent songs.

Unlike the original MM algorithm, **our method does not claim guaranteed optimality**, since playlist sequencing includes musical heuristics, uniqueness constraints, diversity goals, and controlled randomness. Instead, we use the meet-in-the-middle principle as a **practical planning strategy**. This allows the algorithm to preserve the global Energy Arc through recursively fixed anchor points, while still optimizing local transitions inside each segment. As a result, the method **avoids purely greedy choices**, reduces the risk of poor leftover transitions, and produces coherent playlists that are both structured and musically smooth.

1. **In-Depth Understanding of Existing Methods and Tools**
   The challenge of automated playlist generation has been approached from several distinct angles in the Music Information Retrieval (MIR) and Recommender Systems (RS) domains.
   - **Similarity-Based and Sequence Models:** Early research primarily relied on audio similarity and metadata to generate playlists, for example Bonnin et al. (2014) [3] mentions the use of GPS location of users as metadata on page 8 in the table of additional information used. Flexer et al. (2008) [4] demonstrated the generation of playlists by defining a start and end song, using spectral similarity (specifically, Kullback-Leibler divergence between Mel Frequency Cepstrum Coefficients, or MFCCs) to create a smooth, interpolating transition between the two anchor points. However, relying solely on similarity often results in overly homogeneous playlists that lack the dynamic variation required for an engaging listening experience (Bonnin et al. (2014) [3] page 8 under section 4.1), also they talked about generating a playlist while we discuss the situation where a playlist is given and we want to find fitting energetic sequences in it. Moreover, Flexer et al. [4] (on page 177 under Table 8) note that finding a proper transition can fail if the database lacks tracks that fit logically between two highly contrasting anchor songs.

   - **Graph Theory and Combinatorial Optimization:** A more sophisticated approach models track sequencing as a graph traversal problem. Bittner et al. (2017) [5] (on page 3 under section 3.1.2) framed the optimal ordering of a playlist as finding the Shortest Hamiltonian Path in a complete weighted graph. In their model, edge weights are determined by Euclidean distances in a feature space that includes tempo (mapped logarithmically), key (mapped via the circle of fifths), and timbre (in section 3.1.1 in Bittner et al.). Because finding an exact Hamiltonian path is NP-complete, greedy algorithms (e.g., HAM-

1, HAM-2) and Traveling Salesman Problem (TSP) approximations are widely used to minimize overall transition costs. By combining this macro-level sequencing with micro-level cue point selection at major structural boundaries, they demonstrated that beat-aligned transitions strongly enhance listener satisfaction and mask otherwise abrupt changes (under section 4, Transitions). However, a major vulnerability in myopic greedy selection is what is known as the **"Greedy Trap"** (or "leftovers" problem). Bittner et al. explicitly documented this limitation, noting that **"an undesirable artifact is the presence of poor track pairings at the tail of the sequencing for HAM-1".** [5] This highlights the strict necessity for algorithms that can look ahead rather than just taking the immediate lowest cost.

- **Reinforcement Learning (RL) and Markov Decision Processes (MDP):** RL has been utilized to adapt to listener preferences dynamically. Liebman et al. (2015) [6] (on page 591 in their Abstract and part 2 Reinforcement learning frame-work) introduced DJ-MC, an agent that models playlist recommendation as an episodic MDP, learning user preferences over both individual songs and song-to-song transitions by representing songs through rich spectral and rhythmic descriptors (under section 3, Modeling). Recently, Spotify developed an Action-Head Deep Q-Network (AH-DQN) trained in a simulated environment to optimize user satisfaction metrics (like completion rates) by sequentially selecting tracks. [1] (under the Abstract section on page 4948 and “contribution” in the Introduction section on page 4949) . These models highlight the absolute importance of sequential awareness, proving mathematically that the pleasure derived from a song is directly affected by its relative position in a sequence (Leibman et al. Under the introduction section).
  While deep RL can be complex to train**, planning algorithms derived from this domain (such as Beam Search or Monte Carlo Tree Search) provide powerful "RL-lite" alternatives** (Leibman et al. Under section 5, DJ-MC)**.** They evaluate partial sequences and look several steps ahead **to avoid dead ends, proving highly effective for sequential decision making.**
- **DJ Automation and Intra-Track Analysis:** Creating a seamless mix requires analyzing the internal structure of songs to find "switch points" or cue points. Zehren et al. (2022) [7] (under sections Rule-Based Approach and Novelty Detection in pages 70-76) used expert DJ rules and statistical novelty detection algorithms (like Foote's checkerboard kernel) to identify structural boundaries, such as downbeats at the start of a musical period, suitable for crossfading. Vande Veire and De Bie (2018) [8] built a complete open-source auto-DJ system for Drum and Bass that automates beat-matching, downbeat tracking, and applies specific crossfade profiles based on structural segmentation. (As

explained in their Abstract and methodology sections, 4 in total and 4.3 and 4.1 specifically on pages 1 & 11-17)

2. **Intelligent Comparison: Evolution and Differences Between Methods**
   The evolution of playlist generation moves from **static similarity retrieval** to **sequential optimization**, and recently toward **adaptive learning (RL).**
   - **Graph/Optimization vs. Reinforcement Learning:** Graph-based approaches (like TSP or Constraint Satisfaction Programming) are highly explainable and allow developers to enforce strict musical rules, such as preventing dissonant harmonic clashes or mandating a specific tempo trajectory [5] (specifying what we talk about in page 1 on introduction and related work, and then on page 5 in sections 4.2 and 4.3 and page 6 on conclusions), They also naturally support the incorporation of explicit mathematical constraints, which is highly beneficial when crafting a deliberate musical journey. However, calculating the cost matrix for large catalogs is computationally expensive ($O(N^2)$) (Bittner et al. Page 3 section 3.1.2). In contrast, RL models (like DJ-MC [6]) excel at personalizing content dynamically based on user feedback (skips/listens) [1] [6] (in Leibman et al. It is mentioned on page 598 on figure 4 explanation and section 9 and on page 595 on section 5.3), (on Tomasi et al. It is on page 4951 on section 4.1). Yet, RL models often act myopically (nearsighted) and struggle to enforce strict long-term structural narratives, such as the classic "Hero's Journey" or "Double Peak" energy arcs used by professional DJs to prevent listener ear fatigue.
     By adopting a **dynamic pipeline**, our system calculates cost functions in real-time, allowing the track pool to be updated on the fly while following the “Energy Arc” and having the start and end songs as context. This mirrors a real live DJ environment, where a DJ responds to the crowd's energy sequentially rather than having the entire night perfectly mapped out in advance.

   - **Inter-track vs. Intra-track Optimization:** While traditional Recommender Systems focus entirely on inter-track mixability (which song should play next), automated DJ research highlights the absolute necessity of intra-track mixability (where exactly the crossfade should occur) Zehren et al. Mentions the other sources we cite as examples for this case [7] (on page 69 under Related Work), noting that switch points profoundly impact the continuous listening experience (under Abstract). Studies show that misaligned beats or transitioning during vocals are primary causes of perceived poor quality in automated mixes, alongside incompatible timbral shifts. Bittner et al. (2017) [5] (pages 5-6 under section 4.3, figure 5, figure 6, table 2, table 3 and conclusions section).
   - **Optimization vs. Diversity:** Greedy algorithms maximize mathematical similarity but destroy musical novelty. By incorporating a probabilistic Random Walk that samples from the Top-K lowest-cost

tracks, developers can balance optimization with critical musical diversity, preventing the algorithm from getting stuck in repetitive loops.

3. **Comparing Existing Solutions to Our Proposed Product** Existing commercial and academic systems largely fail to integrate the macro-level planning of a DJ set with the micro-level execution of smooth audio transitions. For example, Spotify's default shuffle or basic recommendation algorithms can create jarring, disjointed experiences because they evaluate tracks independently. While advanced RL models aim to maximize listen time [1] (Tomasi et al. On page 4952 under reward and user simulator, and page 4954 under section 5.2), they do not explicitly plan an "Energy Arc" (e.g., Ramp-Up for a warmup set, or Rollercoaster for a peak-time party).
Our proposed product tackles this directly through a **Context-Aware Random Walk with Beam Search.** At each time step **t**, the system computes a cost function **J** for all remaining tracks in the pool, evaluating their harmonic compatibility and proximity to the target energy curve. To prevent the "Greedy Trap" observed in previous literature [5] (Bittner et al. Under section 3.1.2), **we utilize Bi-directional Beam Search to evaluate the next several tracks (lookahead paths) before making a selection, choosing the shortest path among future trajectories.** We will rigorously measure the success of this model using two primary metrics: **1) The convergence to the energy arc (measured via RMSE between the target curve and the generated sequence), and 2) Sequential Coherence.** As Schweiger et al. (2025) [9] (Abstract and Introduction) highlight, traditional coherence metrics are flawed; therefore, we will apply their formal definition for measuring playlist coherence **"based on the sequential ordering of tracks",** mathematically evaluating the order-dependent smoothness of BPM and harmonic transitions over time.
4. **Relevant Papers, Tools, and Resources for Citation & Extraction** To build and evaluate Our system, we would utilize the following existing tools and datasets:
    - **Audio Feature Extraction Tools** (for future work and not the first part research we are doing in the scope of the course):
        - **Librosa:** The standard Python library for Music Information Retrieval (MIR). Excellent for extracting Mel-spectrograms, MFCCs, beat tracking, and onset detection.
        - **Essentia:** A highly optimized C++ library with Python bindings. It is heavily recommended over Librosa for processing large datasets quickly (up to twice as fast for tasks like FFT) and extracting advanced psychoacoustic features.
    - **Open Datasets** (what we will actually use in our project):
        - **FMA (Free Music Archive) & MTG-Jamendo:** Due to recent closures of Spotify's Audio Features API, these open datasets are critical. They provide hundreds of thousands of tracks with pre-computed acoustic features, tags, and completely open audio files suitable for rendering crossfades.

- **Million Playlist Dataset (MPD):** Useful for analyzing human curation patterns and sequence coherence. Also mentioned to be used on Schweiger et al. (2025) [9] research on user-curated playlists.
- **UnmixDB:** Contains ground-truth annotations of DJ mixes, cue points, and beat tracking, which is essential if you plan to implement automatic crossfading and switch-point detection. Also mentioned to be used on Zehren et al. (2022) [7].

5. **Proposed Algorithm – Recursive Bidirectional Energy-Aware Beam Search:**

   We define the playlist sequencing task as a constrained ordering problem. Given a fixed pool of songs:

   $$S = \{s_1, s_2, \dots, s_N\}$$

   a required start song s_{start}, a required end song s_{end}, and a target energy arc $A(t)$, the goal is to construct an ordered playlist:

   $$P = (p_1, p_2, \dots, p_N)$$

   such that:

   $$p_1 = s_{start}, p_N = s_{end}$$

   and every song from the original pool appears exactly once.

   Our proposed method, **Recursive Bidirectional Energy-Aware Beam Search**, is inspired by bidirectional heuristic search, especially the MM algorithm, which searches from both the start and the goal and is designed to meet near the middle of the solution path (Holte et al. 2016 [2]). However, unlike MM, our method is not intended to guarantee an optimal shortest path. Instead, it adapts the meet-in-the-middle principle to the musical sequencing problem, where the objective includes global energy structure, local transition quality, and controlled diversity.

   And we will expand more about the algorithm in the next chapter.

Works Cited

[1] F. Tomasi, J. Cauteruccio, S. Kanoria, K. Ciosek, M. Rinaldi and a. Z. Dai, "Automatic Music Playlist Generation via Simulation-based Reinforcement Learning," *KDD '23: Proceedings of the 29th ACM SIGKDD Conference on Knowledge Discovery and Data Mining,* pp. 4948-4957, 2023.

[2] R. C. Holte, A. Felner, G. Sharon and N. R. Sturtevant, "Bidirectional Search That Is Guaranteed to Meet in the Middle," *30th AAAI Conference on Artificial Intelligence, AAAI 2016,* pp. 3411-3417, 2016.

[3] G. BONNIN and D. JANNACH, "Automated generation of music playlists: Survey and experiments," *ACM Computing Surveys (CSUR),* vol. 47, no. 2, pp. 1-35, 2014.

[4] A. Flexer, D. Schnitzer, M. Gasser and G. Widmer, "PLAYLIST GENERATION USING START AND END SONGS," *ISMIR 2008, 9th International Conference on Music Information Retrieval, Drexel University,* pp. 173-178, 2008.

[5] R. M. Bittner, M. Gu, G. Hernandez, E. J. Humphrey, T. Jehan, P. H. McCurry and N. Montecchio, "AUTOMATIC PLAYLIST SEQUENCING AND TRANSITIONS," *ISMIR 2017: Proceedings of the 18th ISMIR Conference, Suzhou, China,* pp. 442-448, 2017.

[6] E. Liebman, M. Saar-Tsechansky and P. Stone, "DJ-MC: A Reinforcement-Learning Agent for Music Playlist Recommendation," *AAMAS 2015,* pp. 591-599, 2014.

[7] M. Zehren, M. Alunno and P. Bientinesi, "Automatic Detection of Cue Points for the Emulation of DJ Mixing," *Computer Music Journal (2022),* vol. 46, pp. 67-82, 2022.

[8] L. V. Veire and T. D. Bie, "From raw audio to a seamless mix: creating an automated DJ system for Drum and Bass," *EURASIP Journal on Audio, Speech, and Music Processing,* vol. 2018, 2018.

[9] H. Schweiger, E. Parada-Cabaleiro and M. Schedl, "The impact of playlist characteristics on coherence in user-curated music playlists," *EPJ Data Science,* vol. 14, 2025.

Algorithm visualization: